UNIVERSIDAD DEL VALLE DE GUATEMALA

CONSTRUCCIÓN DE COMPILADORES

Pablo



Intérprete SQL con archivos

Ihan Marroquin - 23108

Diego Patzan - 23525

GUATEMALA, 01 de noviembre del 2026

Informe final

https://github.com/UVG-Tareas-y-proyectos/SQL_compis

Arquitectura del intérprete SQL

Desarrollamos un intérprete que recibe SQL, valida su estructura y significado, ejecuta las operaciones y conserva los datos en archivos. Reutilizamos nuestros generadores YALex y YAPar del proyecto anterior para construir el lexer y el parser de este lenguaje. La ejecución y el almacenamiento son propios; no utilizamos un DBMS para procesar las consultas.

```
SQL -> tokens -> árbol sintáctico concreto -> AST
    -> análisis semántico -> interpretación -> archivos -> resultado
```

El lexer utiliza un autómata determinista para reconocer identificadores, palabras reservadas, números, cadenas y operadores. El parser SLR aplica las producciones de SQL.yalp y conserva un árbol concreto con las reducciones y los tokens. La gramática se verifica sin conflictos SLR. El visitador ASTBuilder elimina los signos y reglas auxiliares para producir nodos que representan acciones como select, insert y create_table.

Componente	**Responsabilidad**
grammar/SQL.yal y SQL.yalp	Definir tokens y producciones de SQL.
frontend.py	Generar lexer y tablas SLR; construir el árbol concreto.
ast_builder.py	Transformar el árbol concreto en un AST independiente.
semantic.py y types.py	Resolver tablas y columnas; comprobar tipos y restricciones.
interpreter.py	Evaluar expresiones, consultar y modificar registros.
storage.py e indexes.py	Persistir el catálogo, datos, posiciones e índices.
pipeline.py	Coordinar análisis, ejecución y diagnósticos.
scripts/ide.py y sqlcompis.py	Exponer la interfaz gráfica y la terminal.

La interpretación recorre directamente el AST. En esta entrega SQL no generamos TAC: los nodos de consulta describen las fuentes, condiciones, proyecciones y grupos que debe evaluar el intérprete. La procedencia de los generadores reutilizados se identifica en PROCEDENCIA.txt.

Lenguaje y análisis semántico

El lenguaje incluye CREATE TABLE, DROP TABLE, CREATE INDEX, DROP INDEX, INSERT, UPDATE, DELETE y SELECT. Las consultas admiten WHERE con AND, OR y NOT, ORDER BY, INNER JOIN, COUNT, SUM, AVG, MIN, MAX, GROUP BY, HAVING y subconsultas con IN, NOT IN, EXISTS, NOT EXISTS y comparaciones escalares. Se implementa el subconjunto solicitado; no se incluyen ALTER TABLE, claves foráneas, LEFT JOIN, UNION, LIMIT ni comandos de transacciones SQL.

Tipo	**Validación**
INT	Número entero; BOOLEAN se mantiene como tipo distinto.
FLOAT	Número finito; permite convertir un INT a FLOAT.
VARCHAR(n)	Cadena de hasta n caracteres.
BOOLEAN	TRUE o FALSE.
DATE	Fecha válida con formato YYYY-MM-DD.

El catálogo contiene las columnas y restricciones de cada tabla. Antes de ejecutar comprobamos que existan las tablas y columnas, que los nombres no sean ambiguos, que las expresiones sean compatibles y que INSERT tenga valores válidos. También validamos los índices y las columnas utilizadas con GROUP BY. Las restricciones NOT NULL, PRIMARY KEY, UNIQUE y DEFAULT se conservan en el esquema; las claves compuestas se declaran a nivel de tabla.

```
CREATE TABLE estudiantes (
  id INT PRIMARY KEY,
  nombre VARCHAR(60) NOT NULL,
  edad INT,
  activo BOOLEAN DEFAULT TRUE,
  ingreso DATE
);
INSERT INTO estudiantes (id, nombre, edad, ingreso)
VALUES (1, 'Ana', 20, '2026-01-15');
```

Cada SELECT crea un ámbito con sus tablas y alias. Una columna sin prefijo se busca en el ámbito más cercano y se rechaza si varias tablas pueden proporcionar ese nombre. Una subconsulta puede consultar su ámbito exterior; conservamos la profundidad y el alias resuelto como atributos del nodo. Así una referencia como e.id dentro de una subconsulta mantiene su relación con la fila externa.

Cada diagnóstico identifica fase, mensaje, línea y columna. Una subconsulta escalar con varias columnas se rechaza durante el análisis. La cantidad de registros depende de los datos y se comprueba al evaluar: más de una fila produce un error semántico antes de aplicar cambios de esa sentencia.

Almacenamiento y uso de índices

Cada base es un directorio. El archivo metadata.json conserva tipos, restricciones, definiciones de índices, cantidad de registros e identificadores internos. Los registros se guardan en CSV y sus posiciones en bytes en un archivo JSON separado. Cada celda del CSV contiene un valor JSON para distinguir NULL de una cadena y conservar comas, comillas, caracteres Unicode y saltos de línea.

```
database/
  metadata.json
  tables/estudiantes.csv
  offsets/estudiantes.json
  indexes/idx_edad.json
```

Cada registro tiene un identificador interno estable llamado __rid. El archivo offsets relaciona ese identificador con el inicio del registro en el CSV. Los índices relacionan un valor de columna con los identificadores de los registros que lo contienen. Al reabrir el mismo directorio se recuperan el esquema, las restricciones, los datos y los índices.

```
CREATE INDEX idx_edad ON estudiantes(edad);
SELECT nombre, edad FROM estudiantes WHERE edad >= 20;
DROP INDEX idx_edad;
```

Para una condición aprovechable, el plan obtiene candidatos del índice y el lector accede a esas posiciones con seek. No se recorre el CSV completo para filtrar una columna indexada. En la demostración, edad >= 20 lee dos de los cuatro registros. El IDE y la opción --plan muestran el índice y la cantidad de filas leídas. Para un rango se comparan las claves del índice; no implementamos un árbol B.

Con AND se pueden intersectar candidatos, incluso cuando sólo una parte de la condición aprovecha el índice. Con OR ambos lados deben permitir un conjunto seguro de candidatos; en caso contrario se recorre la tabla. Las referencias entre columnas y las expresiones no aprovechables se evalúan después. También se pueden aprovechar índices en subconsultas correlacionadas cuando el valor buscado proviene de la fila exterior.

INSERT, UPDATE y DELETE reconstruyen los índices afectados junto con los registros y sus posiciones. Las escrituras validan tipos y unicidad antes de guardar. Un diario conserva las versiones anteriores durante el reemplazo de archivos y permite recuperar una escritura interrumpida. Cada sentencia se aplica completa; un script no constituye una transacción que revierta las sentencias anteriores ya completadas.

El uso previsto es una aplicación por directorio de base de datos. Las sesiones del mismo proceso comparten un bloqueo, pero no implementamos coordinación de escritores entre procesos independientes. Los datos se administran desde SQL y no mediante edición manual de los archivos.

Evaluación de consultas y subconsultas

SELECT obtiene filas de sus fuentes, combina las que satisfacen INNER JOIN y filtra con WHERE. Si hay GROUP BY, reúne las filas con la misma clave y calcula las agregaciones por grupo. HAVING filtra esos grupos. La proyección calcula las columnas de salida, DISTINCT elimina duplicados y ORDER BY ordena el resultado.

```
SELECT e.nombre, c.nombre AS curso, i.nota
FROM estudiantes e
INNER JOIN inscripciones i ON e.id = i.estudiante_id
INNER JOIN cursos c ON i.curso_id = c.id
ORDER BY e.nombre, c.nombre;
```

La combinación se implementa comparando filas de las fuentes y evaluando ON. Los alias e, i y c permiten resolver cada columna. El ejemplo produce cinco inscripciones al ejecutar la demostración completa; el resultado incluye el nombre del estudiante, el curso y su nota.

```
SELECT curso_id, COUNT(*) AS cantidad, AVG(nota) AS promedio
FROM inscripciones
GROUP BY curso_id
HAVING COUNT(*) >= 1
ORDER BY curso_id;
```

NULL representa un valor ausente. WHERE conserva sólo condiciones verdaderas; una comparación con NULL produce un valor desconocido. Se usa IS NULL o IS NOT NULL para comprobarlo. COUNT(*) cuenta filas, COUNT(columna) omite nulos y SUM, AVG, MIN y MAX omiten nulos y devuelven NULL si no hay valores. La división devuelve un número de punto flotante. ORDER BY coloca nulos primero en ASC y al final en DESC.

```
SELECT e.nombre FROM estudiantes e
WHERE EXISTS (
  SELECT i.estudiante_id FROM inscripciones i
  WHERE i.estudiante_id = e.id AND i.nota >= 90
);
```

EXISTS pregunta si la consulta interna produce alguna fila. En este caso se evalúa usando el estudiante de la fila exterior; el resultado de la demostración es Ana. IN compara un valor con los resultados de una columna y una consulta escalar aporta un solo valor. Si la escalar no produce filas, aporta NULL; si produce más de una, se reporta el error.

```
SELECT nombre FROM estudiantes
WHERE edad > (SELECT AVG(edad) FROM estudiantes);
```

Instalación y ejecución

Se requiere Python 3.10 o posterior con Tkinter. Desde la carpeta del repositorio se crea el entorno y se instalan las dependencias. El intérprete utiliza la biblioteca estándar; pytest se utiliza para comprobar el proyecto. El primer inicio genera y guarda el lexer y las tablas del parser en generated.

```
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe scripts\ide.py
```

En el IDE, Demo nueva crea una base vacía independiente y carga el ejemplo completo. Ejecutar o F6 procesa el script y muestra los resultados de sus 17 sentencias. El selector de Resultados permite revisar cada sentencia. Tokens, Árbol sintáctico, AST y Errores muestran las etapas del análisis. Plan de acceso permite verificar las filas leídas y los índices utilizados.

Analizar o F5 valida sin ejecutar. Si una consulta falla, se limpian los resultados anteriores y se selecciona la pestaña de errores. Al seleccionar un diagnóstico, el cursor se mueve a su ubicación. Base de datos permite continuar con un directorio existente; la ruta activa aparece sobre el editor. Para repetir el script completo se utiliza otra Demo nueva, porque sus tablas ya existen después de la primera ejecución.

La terminal permite ejecutar el mismo ejemplo y consultar después la base persistida. La carpeta de la primera ejecución debe estar vacía.

```
.\.venv\Scripts\python.exe scripts\sqlcompis.py `
  examples\demo_completa.sql --db database\demo1 --plan
.\.venv\Scripts\python.exe scripts\sqlcompis.py `
  --db database\demo1 -c "SELECT nombre FROM estudiantes;"
```

La opción --analizar valida sin ejecutar ni modificar los datos. --tokens, --arbol y --ast muestran las representaciones del análisis; se combinan con --analizar para una revisión sin ejecución. --json produce una salida estructurada. La terminal devuelve 0 al terminar correctamente, 1 ante errores de análisis o ejecución y 2 si no puede abrir el archivo solicitado.

Pruebas y ejemplos incluidos

```
.\.venv\Scripts\python.exe -m pytest -q
```

Las 102 pruebas verifican generación sin caché, precedencia, CST y AST, ubicación de errores, tipos, restricciones, persistencia, recuperación de escrituras, consultas, JOIN, grupos, nulos, subconsultas e índices. Se compara el resultado con y sin índice y se comprueba que una consulta indexada solicite al lector únicamente los registros candidatos. También se prueban la CLI en procesos nuevos y la limpieza del IDE después de un error.

examples contiene el esquema, los datos y consultas separados por tema, además de demo_completa.sql. Los archivos error_sintactico.sql, error_semantico.sql y error_subconsulta.sql permiten comprobar los diagnósticos. Los dos últimos requieren cargar antes las tablas de la demostración. Las pruebas automatizadas utilizan directorios temporales y no modifican las bases de trabajo.